WCRC 2026 · 2차 교육 · 개선 실습

Mission2B TODO 구현 방안

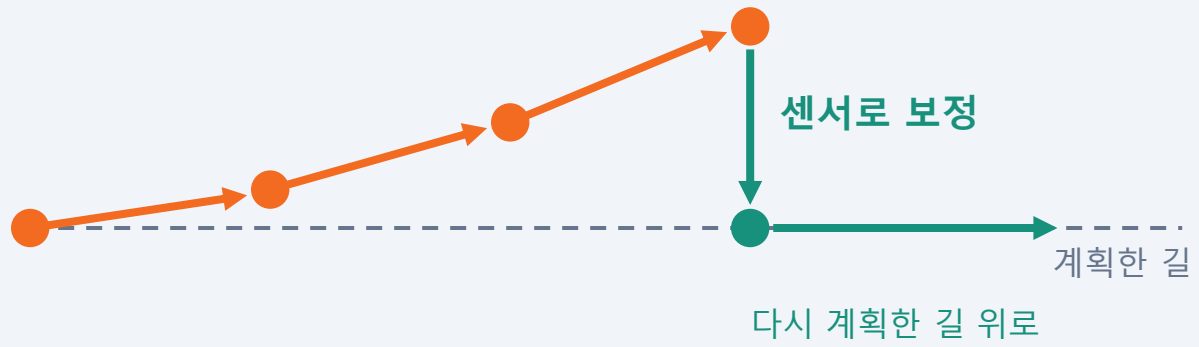
코드에 표시된 18곳에
무엇을, 왜, 어떻게 넣는지 알아봅니다



집기와 낙하	01~02
B로 가는 길	03~04
B에서 복귀	05~08
A 운반과 종료	09~12
적재함 탐색	13~18

Mission2B 변경 내용 수업을 들은 참가 학생용 · 코드는 직접 작성합니다

기본 코드는 바퀴만 믿고 달리는 구간이 많습니다



18곳 코드에 TODO로 표시된 자리

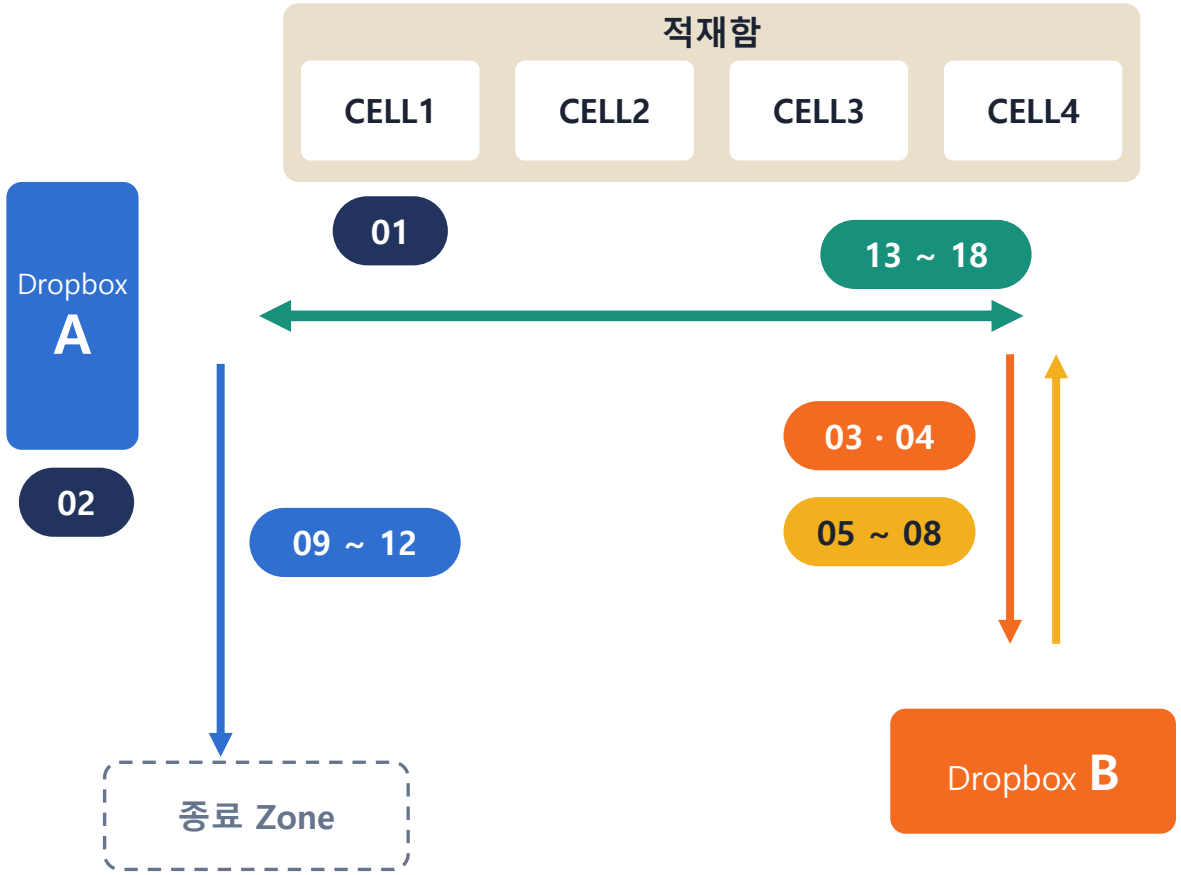
- 1 **카메라 좌우 보정 반복** — TODO-01
- 1 **낙하 자세 시간 조정** — TODO-02
- 16 **PSD 보정 추가** — TODO-03 ~ 18

보정을 넣는 자리의 원칙

오차가 쌓인 직후(긴 이동, 회전 뒤) 그리고 **정확해야 하는 동작 직전**(집기, 낙하, 긴 후진 앞)

전체 그림

TODO 18곳은 다섯 구역에 있습니다



코드의 이동 방향과 거리로 그린 개념도입니다. 실제 크기 비율과는 다릅니다.

13 ~ 18 — 적재함 탐색
탐색선과 각 CELL 앞에서 정면을 바로잡습니다.

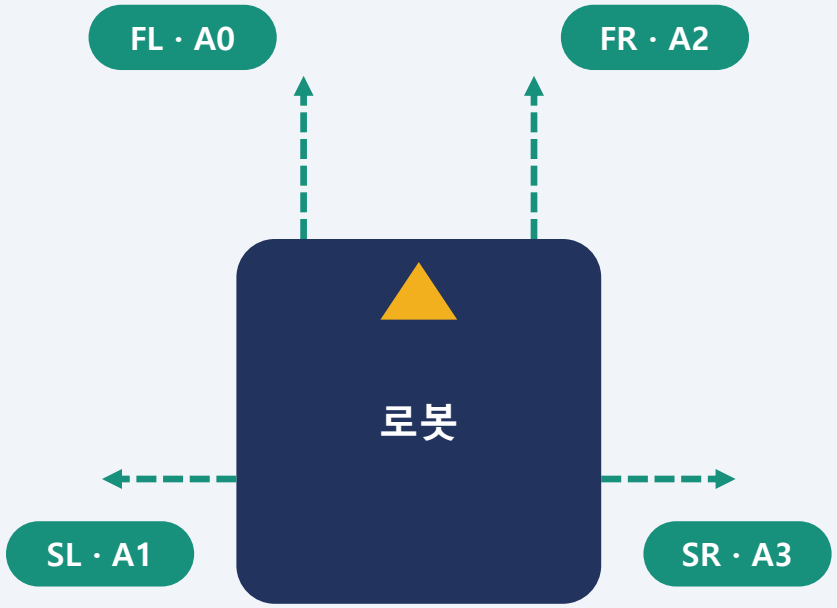
09 ~ 12 — A 운반과 종료
긴 좌진의 중간, 낙하선과 종료선에서 맞춥니다.

03 · 04 — B로 가는 길
회전한 직후와 B 정면에서 거리와 비틀림을 맞춥니다.

05 ~ 08 — B에서 복귀
돌아오는 길에 CELL2 앞에서 네 번 맞춥니다.

01 · 02 — 집기와 낙하
카메라 좌우 보정을 반복하고, 낙하 자세 시간을 늘립니다.

보정 도구 4가지를 먼저 익힙니다



노란 삼각형이 로봇의 앞입니다. A0~A3은 연결된 핀입니다.

FR `PerformFrontMoveAlign( 목표, 허용 )`

앞 오른쪽 센서 하나로 **앞뒤 거리**를 맞춥니다.

SL `PerformLeftMoveAlign( 목표, 허용, 속도상한 )`

왼쪽 센서로 **좌우 거리**를 맞춥니다. 속도상한은 생략할 수 있습니다.

SR `PerformRightMoveAlign( 목표, 허용, 반복, 속도, 제어비 )`

오른쪽 센서로 **좌우 거리**를 맞춥니다.

FL + FR `PerformMoveRotateAlign( 평균목표, 이동허용, 회전허용, 차이목표 )`

앞 센서 두 개로 **앞뒤 거리와 비틀림**을 함께 맞춥니다.

PSD 목표값은 mm가 아니라 센서 값입니다

1



가까울수록 값이 커집니다

Dropbox A 옆 탐색선의 목표는 SL 558입니다. 14mm 더 다가간 낙하선의 목표는 632입니다.

2

목표값은 내 로봇에서 잽니다

로봇을 원하는 자리에 놓고 psdMon으로 읽은 값을 코드에 적습니다. 센서마다 값이 조금씩 다릅니다.

3

끝났다고 맞은 것은 아닙니다

한 번 부르면 최대 15회(3 × 5) 재고 움직이고, 3초가 지나면 끝냅니다. 중요한 곳은 두 번 부릅니다.

목표값과 허용 오차 읽기

FR 보정: 목표 **315**, 허용 오차 **2** → 앞 오른쪽 센서(FR) 값이 **313~317**이 되도록 로봇을 앞뒤로 움직입니다.

앞 센서 두 개로 거리와 비틀림을 함께 맞춥니다



로봇이 왼쪽으로 틀어져 있으면 오른쪽 센서가 면에 더 가깝습니다. 가까운 쪽 값이 더 큽니다.

거리 **이동 오차 = 평균목표 − (FL + FR) ÷ 2**

0에 가까워지도록 앞뒤로 움직입니다.

비틀림 **회전 오차 = (FR − FL) − 차이목표**

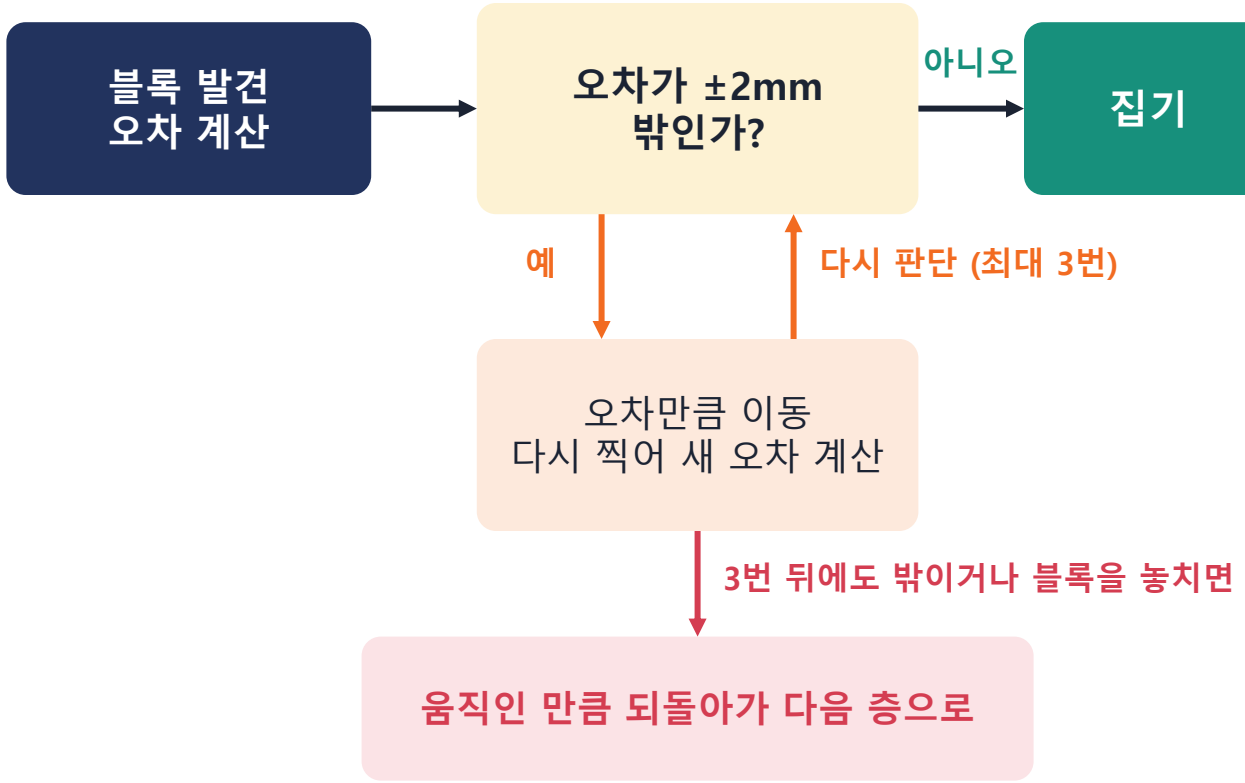
0에 가까워지도록 제자리에서 돕니다.

예) 평균목표 315, 차이목표 8

평균 = (300 + 320) ÷ 2 = 310 → 이동 오차 **+5** (조금 앞으로)
차이 = 320 − 300 = 20 → 회전 오차 **+12** (돌아서 바로잡기)

차이목표가 0이 아닌 이유: 두 센서의 값이 조금씩 달라서, 똑바로 섰을 때의 FR − FL 값을 미리 재서 넣습니다.

카메라 좌우 보정을 최대 3번까지 다시 시도합니다



기본 코드

한 번 움직이고, 허용 범위 밖이면 포기

TODO-01

새 좌표로 다시 계산해 **최대 3번**. 맞으면 바로 집기

예시

1회	오차 +6.0mm	→ **오른쪽 6.0mm**
2회	오차 −2.4mm	→ **왼쪽 2.4mm**
3회	오차 +0.6mm	→ **범위 안, 집기**

TODO-01 · 집기

새 좌표로 다시 계산하며 반복합니다

구현 순서

1. **반복 횟수를 정한다**
   카메라 설정에 최대 반복 횟수 **3**을 상수로 추가합니다.

2. **변수 세 개를 준비한다**
   오차(고쳐 쓸 수 있게), 움직인 양의 합계, 블록을 다시 찾았는지 여부.

3. **반복 조건을 건다**
   3번 미만이고, 오차가 아직 허용 범위(±2mm) 밖인 동안만 반복합니다.

4. **한 번 돌 때 할 일**
   오차만큼 좌우로 이동 → 합계에 더하기 → 0.1초 대기 → 다시 촬영.

5. **새 오차를 계산한다**
   블록이 보이면 새 좌표로 다시 계산하고, 안 보이면 반복을 멈춥니다.

이미 있는 재료

좌우로 이동하기

`DriveDistanceSmoothWithInternalAcceleration()`

다시 촬영해 좌표 얻기

`CamFindBlockAndGetStablePosition()`

허용 범위 2mm

`PIXY_TOLERANCE_MM`

화면 px를 mm로

`SETPOINT_X_PX_TO_MM`

주의

오차 변수는 반복하며 고쳐 써야 합니다. 값을 바꿀 수 없게 선언되어 있으면 바꿀 수 있게 고칩니다.

TODO-01 · 집기

실패하면 움직인 만큼만 되돌아갑니다

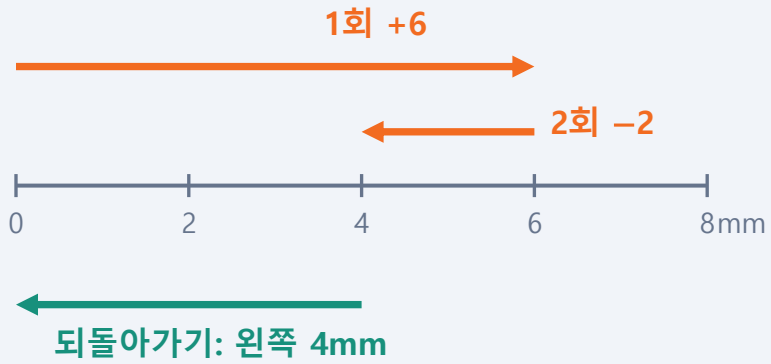
구현 순서

1. **반복이 끝난 뒤 결과를 확인한다**
   블록을 놓쳤거나 오차가 아직 허용 범위 밖이면 실패입니다.
2. **실패했다면 되돌아간다**
   움직인 양의 합계가 0이 아니면, 반대 방향으로 그만큼 이동합니다.
3. **다음 층으로 넘어간다**
   이 층은 포기하고 다음 층을 찾습니다.
4. **성공했다면 집는다**
   허용 범위 안이면 원래대로 집기 동작을 합니다.

기본 코드와 다른 점

기본 코드는 한 번만 움직였으므로 그 값만큼 되돌아갔습니다. 여러 번 움직이면 마지막 오차가 아니라 **이동량의 합계**가 필요합니다.

움직인 양을 부호와 함께 더합니다



합계 = (+6) + (−2) = +4

합계가 +이면 왼쪽으로, −이면 오른쪽으로 그만큼 돌아갑니다.

멀리 뻗는 칸은 낙하 자세 시간을 더 줍니다

Dropbox A의 칸별 낙하 자세 시간

A1 700ms 기본 + 200	**A2** 500ms 기본	**A3** 500ms 기본	**A4** 700ms 기본 + 200
A5 600ms 기본 + 100	**A6** 500ms 기본	**A7** 500ms 기본	**A8** 600ms 기본 + 100

칸 배치는 추첨 화면의 Zone A 배치와 같습니다. B의 칸은 모두 500ms 그대로입니다.

직접 확인

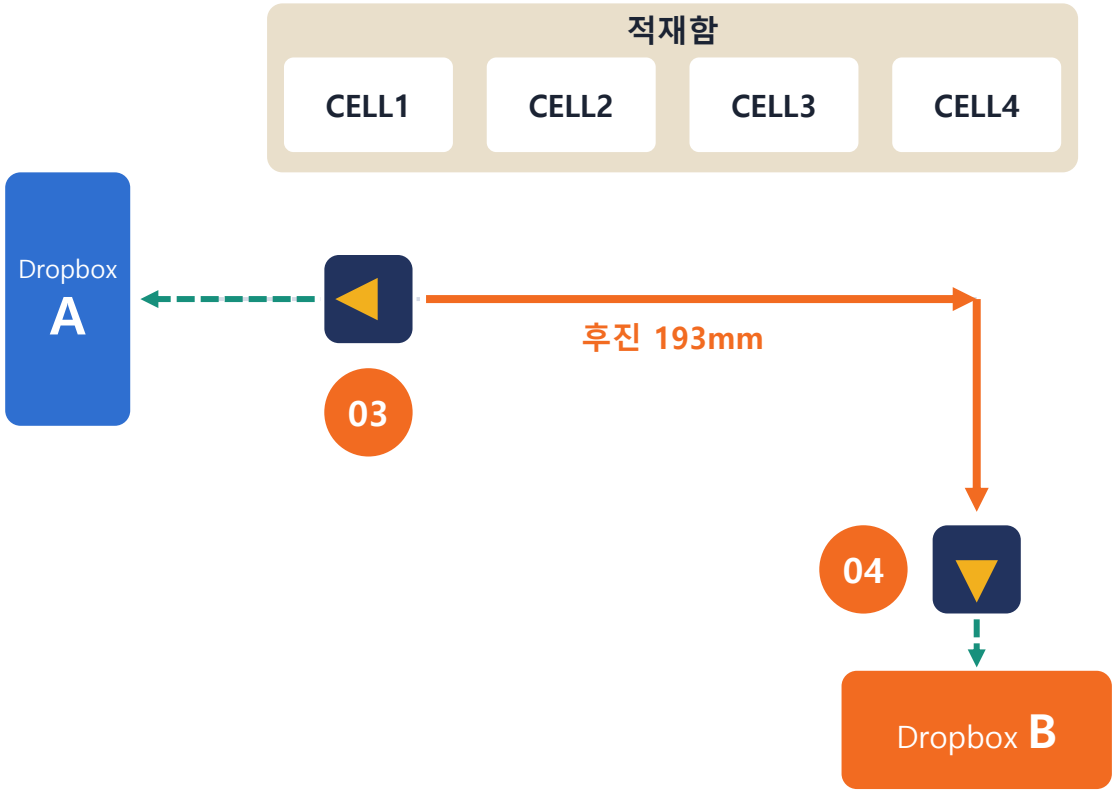
낙하 자세 시간 값을 시리얼 모니터에 출력해 보세요. A1은 **700**, A5는 **600**, A2는 **500**이 나오면 맞습니다.

1. **시간을 변수로 바꾼다**
   시간을 담는 변수를 만들고 500ms를 넣습니다.
2. **A일 때 칸에 따라 더한다**
   A1, A4 칸은 +200ms, A5, A8 칸은 +100ms.
3. **두 곳 모두 변수를 쓴다**
   자세 명령의 시간과, 그 뒤에 기다리는 시간입니다.

확인 포인트: 칸 번호는 어디에 있나요?

칸 번호는 `destination` 에 있습니다. A는 **1~8**입니다. `dropPoseId` 는 EEPROM **자세 번호**(A는 11~18)라서, 칸 번호와 비교하면 조건이 맞지 않습니다.

회전한 직후와 B 앞에서 자세를 바로잡습니다



CELL1에서 집은 경우. 청록 점선은 센서가 보는 방향입니다.

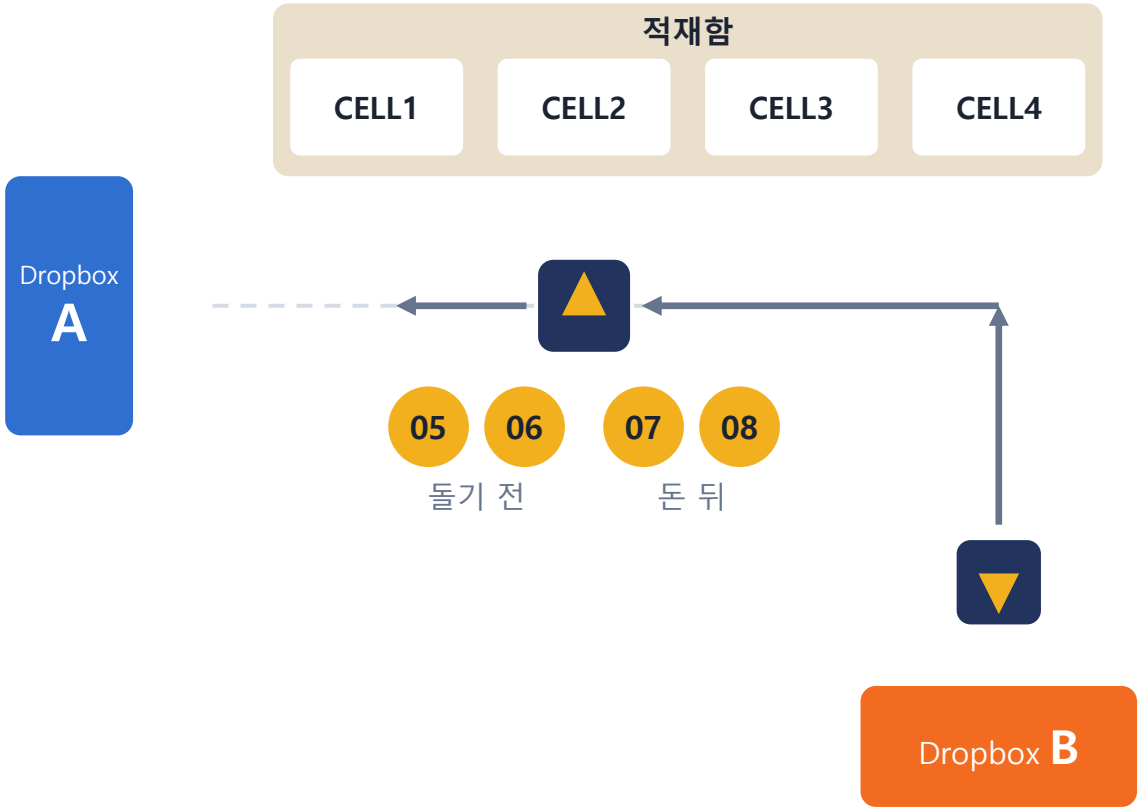
03 1차 좌회전 직후, CELL1에서 집었을 때만

쓸 함수	`PerformMoveRotateAlign`
목표값	평균 **285**, 차이 **13** (정면의 Dropbox A 기준)
허용 오차	이동 2, 회전 2
조건	집은 열이 CELL1일 때만 실행합니다

04 전진 366mm 뒤, 낙하 자세 전

쓸 함수	`PerformMoveRotateAlign`
목표값	평균 **591**, 차이 **6** (B의 면 기준)
허용 오차	이동 2, 회전 2
쓸 상수	`PSD_DROPBOX_B_` 로 시작하는 상수 네 개

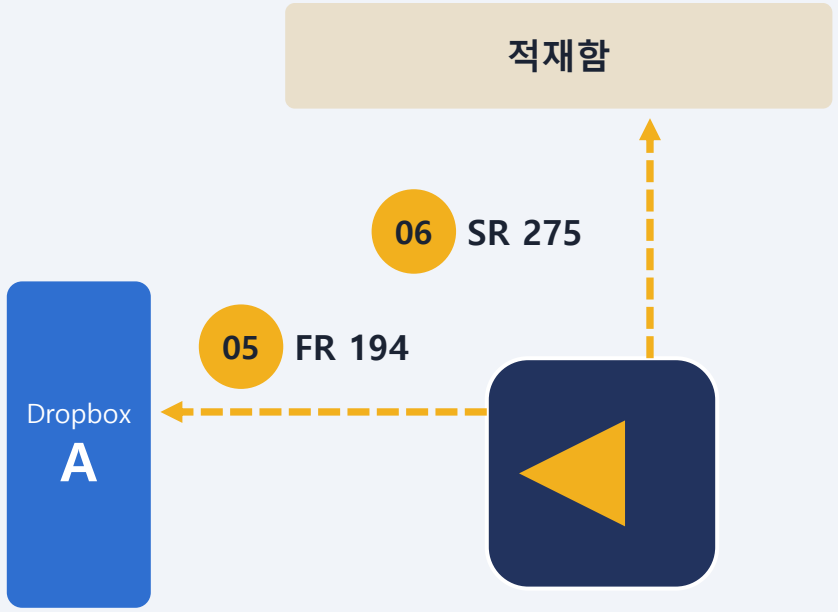
돌아오는 길, CELL2 앞에서 네 번 맞춥니다



	후진 366mm, 우회전, 전진 124mm	
05	**앞 오른쪽(FR)으로 A까지 거리**	194, 두 번
06	**오른쪽(SR)으로 적재함까지 거리**	275
	우회전, 탐색 자세 준비	
07	**앞 두 개(FL, FR)로 적재함 정면**	평균 311, 차이 8
08	**왼쪽(SL)으로 A까지 거리**	200
	좌진 67mm, CELL1 도착	

회전 2번과 이동 3번의 오차가 모이는 자리입니다. 다음 블록 탐색의 출발점을 여기서 다시 잡습니다.

돌기 전에: 앞쪽과 오른쪽 거리를 맞춥니다



넣을 자리: 전진 124mm 뒤, 두 번째 우회전 전. 로봇은 Dropbox A 쪽을 보고 있습니다.

05 앞 오른쪽 PSD로 Dropbox A와의 거리

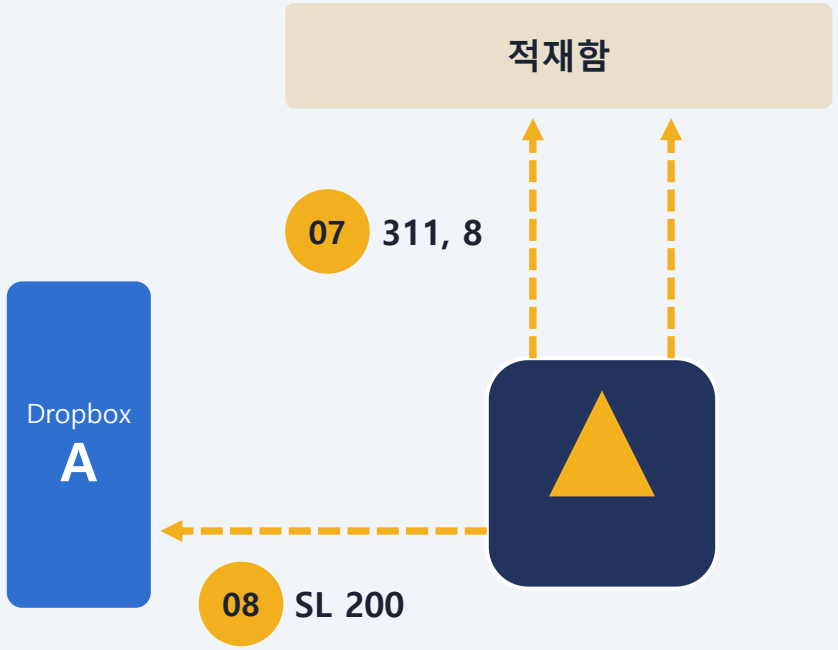
쓸 함수	`PerformFrontMoveAlign` **두 번** (50ms 간격)
목표값	FR **194**
허용 오차	2

06 오른쪽 PSD로 적재함과의 거리

쓸 함수	`PerformRightMoveAlign`
목표값	SR **275**
허용 오차	2
나머지 값	반복 5회, 속도 160, 제어비 0.01을 모두 적습니다

속도를 생략하면 기본값 100이 쓰입니다.

돈 뒤에: 적재함 정면과 왼쪽 거리를 맞춥니다



넣을 자리: 두 번째 우회전과 탐색 자세 준비 뒤. 로봇은 적재함을 보고 있습니다.

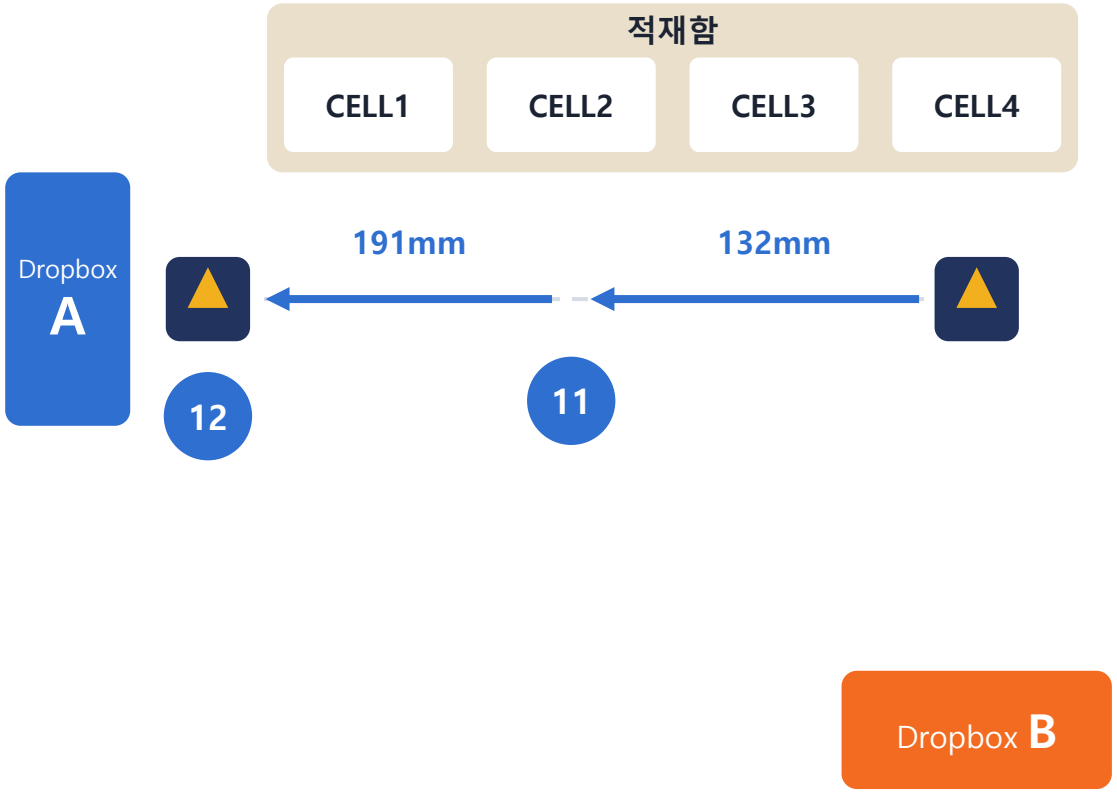
07 앞 센서 두 개로 적재함 정면의 거리와 비틀림

쓸 함수	`PerformMoveRotateAlign`
목표값	평균 **311**, 차이 **8**
허용 오차	이동 2, 회전 2
주의	탐색 중 CELL 앞에서 쓰는 315와 다른 값입니다

08 왼쪽 PSD로 Dropbox A와의 거리

쓸 함수	`PerformLeftMoveAlign`
목표값	SL **200**
허용 오차	2
다음 동작	67mm 좌진해 CELL1에 도착합니다

먼 길은 중간에 한 번, 낙하선에서 한 번 더



CELL4에서 집어 Dropbox A로 가는 경우

11 CELL4에서 집어 A로 갈 때만

조건	집은 열이 CELL4일 때만 (다른 열은 원래대로)
① 좌진	먼저 **132mm** (CELL4에서 CELL2까지)
② 보정	`PerformMoveRotateAlign` 평균 **315**, 차이 **8**
③ 좌진	나머지 **191mm** (전체 323 − 132)

12 모든 열에서, A 낙하 직전에

쓸 함수	`PerformLeftMoveAlign`
목표값	SL **632**
허용 오차	3
속도 상한	100 (Mission2에 있던 보정을 다시 넣습니다)

끝내러 갈 때도 두 번 맞춥니다



넣는 곳: `FinishFromStorageColumn()`

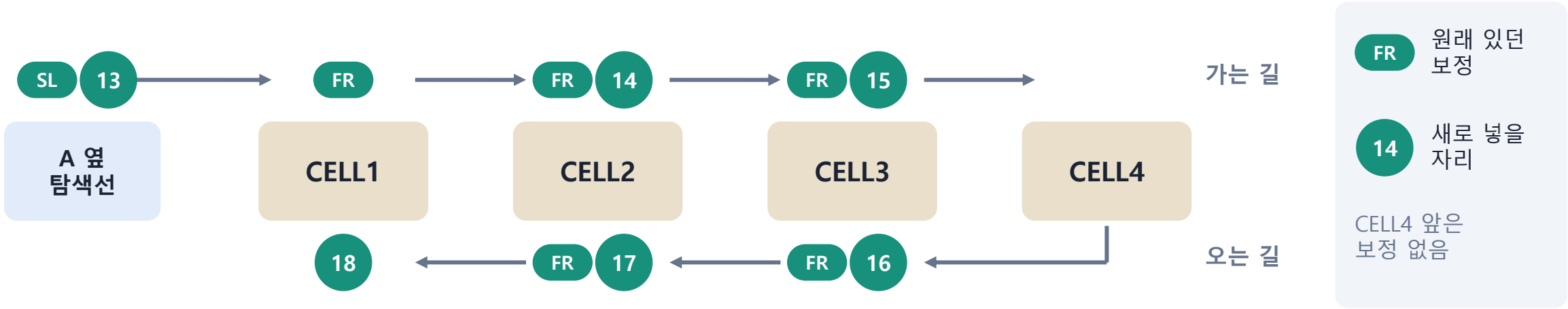
09 팔을 접기 전, CELL1 앞일 때만

쓸 함수	`PerformFrontMoveAlign`
목표값	FR **315**
허용 오차	2
조건	CELL1 앞에서 끝낼 때만 실행합니다

10 종료선까지 좌진한 뒤, 후진 직전에

쓸 함수	`PerformLeftMoveAlign`
목표값	SL **632**
허용 오차	3
속도 상한	100 (TODO-12와 같은 값, 다른 자리)

탐색 구간은 칸마다 정면을 바로잡습니다



TODO	자리	넣을 것
13	탐색선에 들어설 때	SL 보정을 두 번으로 (SL 558)
14 ~ 17	CELL2와 CELL3 앞, 가는 길과 오는 길	FR 보정 뒤에 앞 센서 두 개 보정 (평균 315, 차이 8)
18	오는 길의 마지막 CELL1	빠져 있던 FR 보정 (FR 315)

처음과 끝: 탐색선과 마지막 CELL1

13 탐색선: SL 보정을 두 번

넣을 자리	`setup()`, CELL1으로 우진하기 전
쓸 함수	`PerformLeftMoveAlign` **두 번** (50ms 간격)
목표값	SL **558**
허용 오차	3

원래 한 번이던 SL 보정을 두 번으로 늘립니다. 858mm를 달려온 직후이고, 여기서 잡은 위치가 CELL1~4의 기준이 됩니다.

18 마지막 CELL1: FR 보정 넣기

넣을 자리	오는 길 CELL1, 블록을 찾기 바로 앞
쓸 함수	`PerformFrontMoveAlign`
목표값	FR **315**
허용 오차	2

가는 길의 CELL1에는 있던 보정이 오는 길의 마지막 CELL1에는 없었습니다. 같은 보정을 한 번 넣습니다.

같은 보정을 네 곳에 넣습니다

넣을 자리	원래 있던 FR 보정 바로 다음, 블록을 찾기 바로 앞
쓸 함수	`PerformMoveRotateAlign`
목표값	평균 **315**, 차이 **8**
허용 오차	이동 2, 회전 2
넣는 곳	**네 곳** 모두 같은 내용

14	**가는 길 CELL2**	67mm 우진 뒤
15	**가는 길 CELL3**	68mm 우진 뒤
16	**오는 길 CELL3**	64mm 좌진 뒤
17	**오는 길 CELL2**	68mm 좌진 뒤

왜 필요한가요?

옆으로 움직이면 로봇이 조금씩 틀어집니다. 틀어진 채로 보면 블록이 화면에서 옆으로 밀려 보이고, 팔도 비스듬히 들어갑니다. 그래서 찍기 전에 정면을 맞춥니다.

값을 확인하세요

평균 목표는 **315**입니다. FR 보정과 같은 상수 `PSD_STORAGE_FR` 을 씁니다. 07번에서 쓰는 311과 다릅니다.

정리

PSD 보정 16곳의 목표값 한눈에 보기

TODO	자리	함수	목표값	허용
03	B로 가는 길, 1차 좌회전 뒤 (CELL1 출발)	`PerformMoveRotateAlign`	평균 285, 차이 13	2 / 2
04	B 정면, 전진 366mm 뒤	`PerformMoveRotateAlign`	평균 591, 차이 6	2 / 2
05	B 복귀, CELL2 앞 (돌기 전)	`PerformFrontMoveAlign × 2`	FR 194	2
06	B 복귀, CELL2 앞 (돌기 전)	`PerformRightMoveAlign`	SR 275	2
07	B 복귀, CELL2 앞 (돈 뒤)	`PerformMoveRotateAlign`	평균 311, 차이 8	2 / 2
08	B 복귀, CELL2 앞 (돈 뒤)	`PerformLeftMoveAlign`	SL 200	2
09	탐색 포기 종료, CELL1 앞	`PerformFrontMoveAlign`	FR 315	2
10	탐색 포기 종료, 종료선	`PerformLeftMoveAlign`	SL 632 (속도 100)	3
11	A 운반 중 CELL2 앞 (CELL4 출발)	`PerformMoveRotateAlign`	평균 315, 차이 8	2 / 2
12	A 낙하선	`PerformLeftMoveAlign`	SL 632 (속도 100)	3
13	탐색선에 들어설 때	`PerformLeftMoveAlign × 2`	SL 558	3
14 ~ 17	탐색 중 CELL2, CELL3 앞	`PerformMoveRotateAlign`	평균 315, 차이 8	2 / 2
18	탐색 중 마지막 CELL1	`PerformFrontMoveAlign`	FR 315	2

제공 코드에 적힌 값입니다. 센서와 바퀴는 로봇마다 다르므로 psdMon으로 다시 재서 넣습니다.

실습

한 번에 하나씩 넣고, 바로 시험합니다

1

적재함 탐색

`TODO 13, 18, 14 ~ 17`

`USE_SHORT_CUT_PASS = true` 로 적재함 앞에서 시작해 반복합니다.

2

A로 운반

`TODO 12, 11, 02`

테스트 배정표의 목적지를 A 칸으로 바꾸고 낙하 위치를 봅니다.

3

B로 가는 길

`TODO 04, 03`

테스트 배정표(B1~B4) 그대로 B 앞 도착 자세를 봅니다.

4

B에서 복귀

`TODO 05 ~ 08`

CELL1에 돌아온 자리를 바닥에 표시해 매번 비교합니다.

5

종료와 집기

`TODO 09, 10, 01`

블록을 빼서 탐색 포기를 만들고, 블록을 치우쳐 놓아 봅니다.

`STOP_BEFORE_DROP` 과 `PAUSE_AT_EACH_STEP` 은 `REAL_WCRC_GAME_MODE = false` 일 때만 동작합니다. **경기 전에는 모든 스위치를 되돌립니다.**

실습

잘 들어갔는지 이렇게 확인합니다

TODO	이런 상황을 만들고	이것을 봅니다
01	블록을 칸의 한쪽으로 치우쳐 놓는다	세 번 안에 집는지, 실패하면 제자리로 돌아오는지
02	A1, A4, A5, A8 칸에 넣는다	자세가 끝난 뒤에 그리퍼가 열리는지, 낙하 자세 시간(700, 600)
03 · 04	CELL1에서 집어 B로 운반한다	회전 뒤 곧게 후진하는지, B의 면과 나란한지
05 ~ 08	B에 넣고 다음 블록을 찾는다	CELL1에 돌아온 자리가 매번 같은지
09 · 10	찾을 블록을 일부러 뺀다	종료 Zone에 도착한 자리
11 · 12	CELL4에서 집어 A로 운반한다	낙하선 위치, 블록이 칸 가운데로 들어가는지
13 ~ 18	왕복 탐색을 여러 번 반복한다	각 CELL에서 블록이 화면 가운데에 오는지

시리얼 모니터에 센서 값을 출력해 **보정 전과 후를 숫자로** 비교하면 가장 확실합니다.

실습

자주 하는 실수 다섯 가지

1	**목표값을 mm로 생각한다**	PSD 센서 값입니다. psdMon으로 읽은 값을 적습니다.
2	**조건을 빼먹는다**	03과 09는 CELL1일 때만, 11은 CELL4일 때만 실행합니다.
3	**네 곳 중 한 곳을 놓친다**	14와 15는 가는 길, 16과 17은 오는 길입니다. 서로 다른 자리입니다.
4	**남은 거리를 잘못 계산한다**	11의 두 번째 좌진은 전체 거리에서 첫 구간을 뺀 191mm입니다.
5	**칸 번호와 자세 번호를 섞는다**	`dropPoseId` 는 자세 번호(11~18, 19~26)입니다. 02에서는 `destination` 으로 비교합니다.

오늘의 정리

01 **보정은 오차가 쌓인 직후, 정확해야 하는 동작 직전에**
긴 이동과 회전 뒤, 집기와 낙하와 긴 후진 앞이 그 자리입니다.

02 **센서에 맞는 함수를 고른다**
앞뒤는 FR, 좌우는 SL과 SR, 비틀림까지는 FL과 FR 두 개.

03 **값은 내 로봇에서 재고, 하나 넣을 때마다 시험한다**
제공 코드의 숫자는 출발점입니다.

실습 시작 적재함 탐색 구간(TODO-13)부터 하나씩 넣어 봅시다